# EE5203 L01 Lab Guide

*Build, flash, debug, and modify Blinky - Basri Erdogan*

**Mission:** Create a Nucleo-F401RE project, make LD2 blink, prove that the program executes in the debugger, and independently change its timing.

## Prepare before class

- ☐ Complete the five-question pre-lab quiz.
- ☐ Bring the Question 5 entry ticket.
- ☐ Confirm VS Code + STM32CubeMX are installed (see the Reference page's toolchain table) or identify that you need a laboratory PC.
- ☐ Bring a USB **data** cable if using your own equipment.

## Learning objectives

By checkoff time, you can:

1. build, flash, run, and debug a Nucleo-F401RE project;
2. distinguish build evidence from hardware evidence;
3. set a breakpoint and inspect a variable;
4. make and explain one controlled code modification.

## Hardware and files

| Item | Required value |
|---|---|
| Board | Nucleo-F401RE |
| Programmer/debugger | On-board ST-LINK USB connection |
| Output | LD2, connected to PA5 |
| Tools | STM32CubeMX (configure/generate) + VS Code (edit/build/debug) |
| Project name | `EE5203_L01_<studentnumber>` |

## The opening demonstration

The session begins with one live demonstration of the complete workflow:

```
create -> configure -> generate -> edit -> build -> flash -> debug -> verify
```

Observe which tool owns each stage: the `.ioc` configuration lives in STM32CubeMX; `main.c`, the build terminal, debug controls, and the variable panels live in VS Code. Record questions; do not copy menu clicks without understanding the stage they belong to.

## Checkpoint A - Foundation

1. Connect the board through the ST-LINK USB connector.
2. In **STM32CubeMX**: create a project for **NUCLEO-F401RE** (Board Selector) and accept the board's default peripheral initialization.
3. Confirm that PA5/LD2 is configured as a GPIO output.
4. In Project Manager set the name, the location, and **Toolchain/IDE = CMake**; generate code.
5. Open the generated folder in **VS Code** and locate the protected `USER CODE` regions in `main.c`.
6. Build the unchanged project with zero errors.

**Evidence for Checkpoint A:** Show the selected board, PA5 configuration, and a successful build message.

## Checkpoint B - Core behavior

Inside the **existing generated** `while (1)` user-code region, add these statements. Do not create a second infinite loop.

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(500);
```

Build, flash, and verify a stable blink. Explain why the physical observation is stronger evidence than the build message alone.

## Checkpoint C - Debugger evidence

Add before the loop:

```
int toggle_count = 0;
```

Increment it after every toggle:

```
toggle_count = toggle_count + 1;
```

1. Place a breakpoint on or immediately after the increment.
2. Start a debug session.
3. Predict the value before pressing **Resume**.
4. Resume twice and record the observed values.

| Observation | Predicted | Observed |
|---|---|---|
| First pause | | |
| Second pause | | |
| Third pause | | |

**Evidence for Checkpoint C:** Show the breakpoint, execution marker, and changing `toggle_count` value.

## Checkpoint D - Individual modification

Each student must make one assigned change and demonstrate it individually.

| Variant | Requirement |
|---|---|
| A | Change the delay to 100 ms |
| B | Change the delay to 250 ms |
| C | Change the delay to 750 ms |
| D | Blink three times quickly, then pause for 1 s |

Before flashing, state the predicted behavior. Change only what is necessary, then collect physical or debugger evidence.

## Acceptance checklist

- ☐ Correct Nucleo-F401RE project selected.
- ☐ Project builds without errors.
- ☐ LD2 exhibits the required core behavior.
- ☐ Breakpoint is reached and `toggle_count` changes as predicted.
- ☐ Individual timing variant is demonstrated.
- ☐ Student explains build versus flash versus run evidence.
- ☐ Code remains inside protected user-code regions.

## Individual oral check

Be prepared for one question and one live change:

- What does a successful build prove?
- What does ST-LINK do?
- Why does the breakpoint pause before or after a particular statement?
- Where should application code be placed, and why?
- What evidence proves your modified firmware is running?

## Submit

Submit the requested source/project, one debugger screenshot, the completed prediction table, and a two-sentence debugging report if a failure occurred.

## If you are stuck

Report expected and observed behavior; read the **first relevant** error; confirm the project, board, ST-LINK, build, and flash; test whether a breakpoint reaches the loop; then show the TA your evidence and smallest next test.